AI & MACHINE LEARNING MENTORSHIP

Curriculum

Four weeks, from raw data to a system you can defend in front of a room.

Format	Four weeks. One session and one assignment per week
Who it is for	Non-technical through to professional engineers. Four depths, same topics
Cost	£0 / $0. Every tool used is on a free tier. No payment card at any point
How it runs	Each week is standalone and can be followed on its own
Assessment	A weekly submission, then a capstone build and a live presentation
What you end up with	A deployed AI system of your own, measured, documented and presented

How The Four Weeks Fit Together

Each week answers one question, in order, because each answer is useless without the one before it.

Week	The question	What you leave with
1	What is true, and how do I know?	A labelled dataset, a measured baseline, and an honest accuracy number
2	Can I build and ship something real?	A deployed application with a frontend, a backend, a database and a model
3	Is it actually any good?	Scores attached to real runs, and evidence one version beats another
4	Can I do all of it myself, and explain it?	Your own system, built end to end, presented to the group

The thread running through all four weeks

Anyone can produce an output. The work that gets paid for is proving the output is correct, knowing when it is not, and being able to say so in front of people who will ask hard questions.

That is why the course starts with measurement rather than models, and ends with a presentation rather than a repository.

What This Field Actually Is

Measured from 889 AI Engineer job listings, January 2026.

This is an AI and machine learning program, and the two words pull in different directions. It is worth being precise about which one the job market is actually buying, because it decides what the next four weeks are spent on.

What these roles are for

What the role does	Share of listings
Production work, shipping systems people use	95.6%
Research	4.4%
Building products driven by language models	about 70%
Platforms, infrastructure and data pipelines	28.5%
Traditional machine learning or deep learning	under 2%
Mentions training a model at all	6.4%
Mentions fine-tuning a model	8.5%

> ### Read that last pair again
>
> Training a model appears in roughly one job in sixteen. Nearly everything else is software engineering with a model somewhere inside it: calling an API, retrieving the right context, handling failure, storing results, shipping it, and proving it works.
>
> That is why this program spends one week on data and measurement, two on building and evaluating a real system, and one on shipping something of your own. It is not an oversight that model training is a small part. It is a small part of the job.

What they actually ask you to know

Percentages are the share of listings naming each skill. Nothing here is a majority on its own, which is the point: no single framework or vendor dominates, so understanding the architecture matters more than any one library.

Area	Skill	Share
Language	Python	82.5%
	TypeScript	23.4%
Working with models	RAG, retrieval over your own content	35.9%
	Prompt engineering	29.1%
	Integrating a language model	25.4%
	Agents	14.4%

Area	Skill	Share
	Fine-tuning	8.5%
Cloud and delivery	AWS	40.1%
	Docker	31.0%
	CI and CD	29.3%
	Kubernetes	29.1%
Application	React	14.8%
	FastAPI	10.7%
	PostgreSQL	9.3%
	Vector databases	10.8%
Classical ML	PyTorch	22.0%
	TensorFlow	12.9%

How much of it is ordinary software engineering

Requirement	Share of AI-first roles
Backend	about one half
Frontend	about one third
Full stack	about one fifth

> ### What this means for you
>
> A person who can train a classifier but cannot ship a service, store its results or prove it works is not employable in this market. A person who can build and ship software, and who understands where a model helps and how to measure whether it did, is.
>
> So this program treats AI engineering as software engineering with models in it. You will write a system prompt and you will also write a schema, a test and a deployment. Both halves are the job.

Source: analysis of 889 unique AI Engineer listings across five technology hubs, January 2026. aishippinglabs.com. *Corroborated against* technovids.com *and* index.dev.

Week 1 · Data, Ground Truth And Baselines

Before any model, the question of what counts as correct.

The landscape

- What an AI engineer actually does day to day, and how the role differs from ML engineer, data scientist, MLOps engineer, AI product manager and research engineer
- The skills, tools and frameworks named in real job descriptions for each of those roles
- Where the work actually sits: far more data handling and evaluation than model building

Working with data

- Finding and judging a dataset. Licences, provenance, and why a popular dataset is not automatically a good one
- Loading and inspecting data. Shape, types, missing values, class balance
- Why a file sorted by category will quietly ruin your sample, and how shuffling fixes it
- Splitting data into train and test, and what data leakage looks like when it happens to you

Ground truth and labelling

- Annotation: labelling items by hand and discovering how much judgement it takes
- Writing a labelling guideline precise enough that two people agree
- Inter-annotator agreement. Raw agreement, and why it flatters you
- Cohen’s kappa: agreement corrected for chance
- Published benchmarks, and what it means when your labels disagree with theirs

A first model, and how to judge it

- Text as numbers: bag of words, TF-IDF, and why frequency alone is misleading
- Logistic regression as a baseline classifier
- Why a simple baseline comes before anything clever, and what it protects you from
- Accuracy, precision, recall and F1, and the question each one answers
- The confusion matrix, and reading failure by category rather than in aggregate
- Why accuracy on an imbalanced dataset is close to meaningless

> **The idea the whole course is built on**
>
> You cannot claim a system works without something correct to compare it against. Everything in the three weeks that follow is this idea applied to larger and messier systems.

Week 2 · Building And Shipping An AI Application

From a model call to a product on the internet.

Working with language models

- How an LLM API call is actually shaped: roles, messages, system prompt, user turn
- Tokens, context windows, and why both cost and failure scale with them
- Temperature, determinism, and when randomness is a bug
- Choosing a model, and what happens when the one you depend on is deprecated

Prompt engineering, properly

- Instruction, constraint and refusal: the three jobs a system prompt does
- Grounding an answer in supplied facts, and forbidding everything outside them
- Hallucination: why it happens, what it looks like in production, and why a confident wrong answer is worse than no answer
- Writing a refusal that is clear rather than evasive
- Testing a prompt rather than admiring it

The three tiers of an application

- Frontend: rendering, state, and talking to an API
- Backend: routes, request lifecycle, server-side secrets
- Database: tables, relationships, primary keys, indexes, and basic SQL
- Why the model key never goes anywhere near the browser
- Designing a schema that still makes sense once there is real data in it

Engineering practice

- Git: commits, branches, pull requests, and reading a diff
- Environment variables, secrets management, and what belongs in version control
- Automated testing, and treating the test suite as the specification
- Deployment: build, environment, serverless functions, cold starts
- Reading an error properly: which layer broke, and asking a specific question rather than a vague one
- Using AI coding assistants to understand unfamiliar code, rather than to avoid understanding it

> **Shipping is a skill, not a formality**
>
> Getting a demo working on your own machine and getting something strangers can open are different problems. The second one is where configuration, secrets, cold starts and permissions all become real.

Week 3 · Retrieval, Evaluation And Observability

Measuring a system that has no single right answer.

Retrieval and RAG

- Why pasting everything into the prompt fails on cost before it fails on quality
- Chunking, and what a bad chunk boundary costs you
- Keyword retrieval with BM25, and the parameters that control it
- Embeddings and vector search, and where each approach beats the other
- Relevance thresholds, and the difference between finding nothing and finding rubbish
- Citations: tying every claim back to the passage it came from
- Extraction from real documents, and what a PDF loses on the way in

Agents and control flow

- Routing a request to the right branch, by rule and by model
- Planning a sequence of steps, and validating the plan before executing it
- Tool calling: schemas, arguments, and handling a tool that fails
- Guardrails: blocking checks, advisory checks, and automatic repair
- Composing an answer tied to what was actually retrieved

Observability

- Tracing: a trace, a span, and the parent relationship that makes a run readable
- OpenTelemetry as the standard that keeps you portable between platforms
- Reading a single run end to end and finding where it went wrong
- Joining a trace to your own database so a score computed later lands on the right run

Evaluation

- Golden sets: building one, grouping it, and why the groups matter more than the total
- Regression testing a non-deterministic system
- Scoring retrieval separately from generation, because they fail for different reasons
- Retrieval metrics: recall@k, MRR, empty rate
- Normalisation, and the correct answer your grader marks wrong on formatting
- LLM as a judge: writing one, and the rubric that makes it repeatable
- **Calibrating a judge against human labels before trusting a single number it produces.** A model grading a model is fast, scalable, and confidently wrong in exactly the places you would most like it to be right
- Judge bias: length, position and self-preference, and how to measure each

Versioning, cost and control

- Prompt versioning: champion against challenger on one fixed test, with the promotion rule written before the results arrive
- Rate limits, backoff and retry, and cost modelled across tokens, observability units and wall clock time
- CI gates: making an evaluation block a merge instead of producing a report nobody reads

Week 4 · Capstone And Presentation

Your own system, built end to end, and defended out loud.

No new concepts are introduced this week. You choose a problem, build the system yourself, measure it with the methods from Week 3, and present it to the group.

Scoping

- Choosing a problem small enough to finish and real enough to matter
- Writing down what success means before you start building
- Deciding what you will not build, and saying so
- Identifying who would use it and what would make them stop

Build

- A working system with a frontend, a backend, stored data and a model in the loop
- Grounded answers, with refusal behaviour when the system does not know
- Retrieval over your own content, where the problem calls for it
- Deployed, on a public link, running for anyone who opens it

Measure

- Your own golden set, grouped by the kind of question it tests
- Separate scores for the parts that fail separately
- A judge for whatever has no single right answer, calibrated against your own labels
- Two versions compared on one fixed test, with the improvement stated as a number
- One honest failure, found and reported rather than hidden

Document

- A README someone else could follow to run it
- An architecture description: the pieces, and why each one is there
- Your evaluation results, including what they do not measure
- Cost and limits at a stated volume

The presentation

Ten minutes, then questions. Slides are optional. A working demo is not.

Minutes	What you cover
1	The problem, and who has it
2	A live demo. Including one question it handles badly
2	How it is built, and the one design decision you would defend hardest
3	How you measured it. The numbers, and what they do not capture
2	What broke, what you would do next, and what you would not build again
Q&A	Expect to be asked why you believe your own numbers

How the capstone is assessed

Criterion	What a strong answer looks like
It works	A stranger can open the link and use it without you in the room
It is grounded	Claims trace back to a source, and it refuses cleanly when it does not know
It is measured	Real numbers from your own evaluation, not an impression that it felt good
You know its limits	You can name the thing that would break it before anyone else does
You can explain it	Every design decision has a reason you can say out loud

> ### The question that decides the grade
>
> Name one thing that would make your system worse for a real user and would not move any number you produced.
>
> Naming your own blind spot honestly is worth more than a perfect score. It is the answer that separates someone who runs evaluations from someone who believes them.

Four depths, one syllabus

Every topic in this document is covered at four levels of depth, so the same week works for someone who has never written code and for someone who ships software professionally.

Nobody is given a reduced version. The non-technical track does the same thinking with a browser and a spreadsheet that the engineering track does with a test suite, and in several weeks it is the harder job.

Tools And Frameworks Covered

All free tiers. Nothing here asks for a payment card.

Area	What you use
Language and notebooks	Python, Jupyter and Colab, TypeScript
Data and classical ML	pandas, scikit-learn, TF-IDF, logistic regression, Cohen’s kappa
Models and inference	Hosted LLM APIs, system prompts, tokens, temperature, deprecation
Application	React and Next.js, FastAPI, REST, server and client boundaries
Data storage	Postgres, schema design, SQL, connection handling
Retrieval	Chunking, BM25, embeddings and vector search, relevance thresholds, citations
Agents	Routing, planning, tool calling, guardrails, composition
Observability	Tracing, spans, OpenTelemetry, run inspection
Evaluation	Golden sets, retrieval metrics, LLM as judge, calibration, bias audit
Engineering	Git and pull requests, environment variables, automated tests, CI, deployment